

PROJECT PROPOSAL

Rancang Bangun Website AI Smart Tutor sebagai Media Tanya Jawab Pembelajaran Interaktif

AI Smart Tutor Website as an Interactive Learning Question-Answering Media

AI Smart Tutor using Advanced Prompt Engineering Techniques

Education Technology | Python Flask | Gemini API | Prompt Engineering

Prepared by:

Ivansyah Putra

Student Name	Ivansyah Putra
Institution / Campus	STMIK Kaputama Binjai
Class / Course	Google Prompt Engineering
Project Type	AI-powered educational chatbot website
Main Technology	Python Flask, Gemini API, HTML, CSS, JavaScript
Data Approach	No database; chat history uses browser Local Storage
Version	Updated proposal aligned with WBS M1-M6

Student / Project Developer

Institution / Campus: STMIK Kaputama Binjai

Class / Course: Google Prompt Engineering

Project: AI Smart Tutor using Advanced Prompt Engineering Techniques

Prepared for the AI-Powered Academic Project deliverables.

1. Executive Summary

This project proposes the development of an AI Smart Tutor website as an interactive learning question-answering media. The website is designed to help students ask academic questions, understand concepts, generate study plans, create quizzes, and evaluate answers using AI-generated responses. The system uses Python Flask as the backend framework, Gemini API as the large language model service, HTML and CSS for the interface, and JavaScript for chatbot interaction.

The project applies advanced prompt engineering techniques by using different prompt templates for several AI Tutor Modes, including Question Answering, Study Plan Generator, Quiz Generator, and Answer Evaluator. Each mode is designed to produce structured, consistent, and educational outputs based on the user selected learning level: Beginner, Intermediate, or Advanced.

This project was developed by Ivansyah Putra, a student from STMIK Kaputama Binjai, for the Google Prompt Engineering class/course. The website implementation, prompt structure, Gemini API integration, chatbot interface, learning level system, project explanation section, and final testing were prepared to support the AI Smart Tutor capstone deliverables.

1.1 Project Developer Identity

2. Background

Digital learning environments increasingly require interactive tools that can support students in understanding academic materials. Traditional learning resources may not always provide instant explanations, personalized guidance, or practice activities. Students often need a system that can answer questions, simplify complex concepts, generate study plans, create quizzes, and evaluate their answers in a structured way.

AI-based educational assistants can help address this need by providing adaptive responses through natural language interaction. By using prompt engineering, AI responses can be controlled and formatted to be more reliable, consistent, and suitable for learning. Therefore, this project focuses on building a web-based AI Smart Tutor that supports interactive learning through a chatbot interface.

3. Problem Statement

- Students need quick and structured explanations for academic topics.
- General AI chatbot responses may be inconsistent without proper prompt control.
- Learning support should adapt to different student levels, such as beginner, intermediate, and advanced.
- Students also need additional learning tools such as study plans, quizzes, and answer evaluation.
- The project must remain lightweight and practical without requiring a database system.

4. Project Objectives

- To develop a web-based AI Smart Tutor as an interactive learning chatbot.
- To integrate Gemini API for generating AI-powered educational responses.
- To provide structured question answering and concept explanation.
- To support multi-level explanations based on Beginner, Intermediate, and Advanced learning levels.
- To provide Study Plan Generator and Quiz Generator features.
- To provide Answer Evaluator functionality with score, strengths, weaknesses, and corrected answer.
- To apply prompt comparison and prompt optimization through mode-based prompt engineering.
- To provide project explanation and documentation directly inside the website.

5. Project Scope

The project scope focuses on building a functional educational chatbot website. The system is designed for academic learning support and does not include user login, online database storage, payment features, or production deployment. The system will run locally using Flask and will use Gemini API for AI response generation.

Included in Scope	Not Included in Scope
AI chatbot web interface	User authentication system
Gemini API integration	Cloud database storage
Question answering mode	Payment or subscription system
Study plan and quiz generation	Mobile application deployment
Answer evaluator mode	Medical, legal, or unsafe content advice
Local Storage for chat history	Large-scale production server deployment

6. Proposed Solution

The proposed solution is an AI Smart Tutor website that allows users to select an AI Tutor Mode, choose a learning level, and type an academic input. The user input is sent to the Flask backend, where a structured prompt is created based on the selected mode and learning level. The backend then sends the prompt to Gemini API and returns the generated response to the chatbot interface.

The website also includes a Project Information section that explains the purpose, features, technologies, workflow, prompt optimization approach, and no-database design. This makes the project easier to present and evaluate during the final demo.

7. Main Features

Feature	Description	Output
Question Answering	Answers academic questions using structured explanation.	Answer, context explanation, concept explanation, and learning tip.
Multi-Level Explanation	Adapts responses based on selected level.	Beginner, Intermediate, or Advanced explanation style.
Study Plan Generator	Creates a learning schedule based on a topic.	Day-by-day study plan and learning guidance.
Quiz Generator	Generates practice questions.	Quiz questions, answer key, and learning tip.
Answer Evaluator	Evaluates user answers like an examiner AI.	Score, strengths, weaknesses, corrected answer, and improvement tip.
Chat History	Stores chat messages without database.	Previous chat remains visible after page refresh.
Project Explanation	Explains project details in the website.	Project overview, technologies, workflow, and prompt optimization.

8. WBS and Milestone Alignment

The website implementation is aligned with the Work Breakdown Structure (WBS) from M1 to M6. The following table summarizes how each milestone is represented in the final system.

Milestone	Requirement	Implementation in Website
M1	Project setup and initial prompt exploration	Flask project structure, Gemini API setup, .env configuration, and basic prompt testing.
M2	Structured answer generator and multi-level explanation generator	Question Answering mode with structured response format and learning level selection.
M3	Study Plan Generator and Quiz Generator	AI Tutor Mode includes Study Plan Generator and Quiz Generator.
M4	Answer Evaluator (Examiner AI)	Answer Evaluator mode provides score, strengths, weaknesses, corrected answer, and feedback.
M5	Prompt Comparison Engine and Optimization	Project uses mode-based prompt templates and includes Prompt Optimization explanation inside the website.
M6	Finalization, Documentation, and Presentation	Website includes final UI design, project explanation section, documentation, proposal, report, slides, and demo preparation.

9. AI Tutor Modes

Mode	Purpose	Sample Input	Expected Output
Question Answering	Answer academic questions and explain concepts.	What is database?	Structured answer and personalized learning support.
Study Plan Generator	Generate a study plan for a topic.	Create a 5-day study plan for HTML and CSS.	Day-by-day study plan.
Quiz Generator	Create practice questions and answer key.	Create 5 quiz questions about information systems.	Five questions and answer key.
Answer Evaluator	Evaluate user answer like examiner AI.	Question: What is database? My answer: ...	Score, feedback, corrected answer, and learning tip.

10. Technology Stack

Technology	Role	Reason for Use
Python Flask	Backend framework	Handles routes, receives chatbot requests, and connects with Gemini API.
Gemini API	AI model service	Generates educational responses based on prompt templates.
HTML	Web structure	Builds the layout and main content of the website.
CSS	Interface design	Creates modern, responsive, and visually clean website styling.
JavaScript	Frontend interaction	Handles message sending, mode selection, UI behavior, and Local Storage.
Local Storage	Client-side history storage	Stores chat history without using a database.

11. Prompt Engineering Approach

This project uses advanced prompt engineering to control the behavior and output quality of Gemini API. Instead of using one general prompt for all requests, the system applies different prompt templates based on the selected AI Tutor Mode. This approach improves response relevance, consistency, and structure.

- Role-based prompting: Gemini is instructed to act as an AI Smart Tutor or Examiner AI.
- Mode-based prompting: each AI Tutor Mode uses a different instruction structure.
- Structured output prompting: the response format is predefined for each mode.
- Learning level adaptation: the explanation style changes based on Beginner, Intermediate, or Advanced level.
- Safety and relevance rules: the AI is guided to answer educational or academic questions only.
- Prompt optimization: prompts are refined to improve clarity, response quality, and task alignment.

AI Tutor Mode	Prompt Focus	Expected Response Improvement
Question Answering	Structured academic explanation	More consistent answer, context explanation, and learning support.
Study Plan Generator	Study schedule generation	Clear day-by-day learning plan.
Quiz Generator	Practice question generation	Organized questions with answer key.
Answer Evaluator	Rubric-based examiner feedback	Score, strengths, weaknesses, corrected answer, and improvement tip.

12. System Workflow

The system workflow starts from the user interface and continues to the backend and Gemini API. The workflow is designed to make the chatbot response match the selected mode and learning level.

1. User opens the AI Smart Tutor website.
2. User selects an AI Tutor Mode.
3. User selects a learning level: Beginner, Intermediate, or Advanced.
4. User types an academic input in the chatbot.
5. JavaScript sends the request to the Flask endpoint.
6. Flask builds a prompt based on the selected mode and level.
7. Flask sends the prompt to Gemini API.
8. Gemini API generates an optimized tutor response.
9. The response is displayed in the chatbot interface.
10. The chat history is saved locally in the browser using Local Storage.

User Selects Mode	Prompt Template Selection	Flask Backend	Gemini API	Optimized Tutor Response
-------------------	---------------------------	---------------	------------	--------------------------

13. Data Storage Approach

This project does not use a database. The reason is to keep the system lightweight, simple, and suitable for a local academic project. Chat history is stored using browser Local Storage, so previous messages can remain visible after refreshing the page. This approach meets the project requirement while avoiding additional database configuration.

14. Testing Plan

Test Case	Input Example	Expected Result	Status
Question Answering	What is database?	AI provides structured explanation.	Planned/Completed
Beginner Level	What is HTML?	AI uses simple words and examples.	Planned/Completed
Advanced Level	Explain database normalization.	AI provides deeper analytical explanation.	Planned/Completed
Study Plan	Create a 5-day study plan for HTML and CSS.	AI generates daily learning plan.	Planned/Completed
Quiz Generator	Create 5 quiz questions about information systems.	AI provides questions and answer key.	Planned/Completed
Answer Evaluator	Question and user answer input.	AI gives score and feedback.	Planned/Completed
Chat History	Refresh page after chat.	Previous chat remains visible.	Planned/Completed
Project Explanation	Click View Project Explanation.	Project information section appears.	Planned/Completed

15. Expected Output

The expected output of this project is a complete AI Smart Tutor website that can be demonstrated through a local browser. The website should display a modern landing page, feature descriptions, a functional chatbot, AI Tutor Mode selection, Learning Level selection, Gemini API responses, chat history, and a project explanation section.

- A web-based chatbot interface in English.
- Structured AI responses based on educational prompt templates.
- Four AI Tutor Modes: Question Answering, Study Plan Generator, Quiz Generator, and Answer Evaluator.
- Three learning levels: Beginner, Intermediate, and Advanced.
- Chat history stored without database.
- Project information section explaining the system and prompt optimization.
- Documentation deliverables including proposal, prompt template library, final report, slides, and demo video.

16. Project Deliverables

Deliverable	Description	Format
Project Proposal	Initial planning document explaining project objectives and approach.	DOCX/PDF
Prompt Template Library	Collection of prompt templates for all AI Tutor Modes.	DOCX/PDF
Final Project Report	Complete final documentation of system design, implementation, testing, and conclusion.	DOCX/PDF
Presentation Slides	Slide deck for project presentation.	PPTX
Prompt Comparison Engine	Optional document or file explaining prompt comparison and optimization.	DOCX/PDF or source file
Demo Video	Video recording showing how the website works.	MP4 or platform upload

17. Conclusion

The AI Smart Tutor project is designed as an interactive educational chatbot website that supports question answering, concept explanation, study planning, quiz generation, and answer evaluation. By using Python Flask, Gemini API, and mode-based prompt engineering, the system can generate structured and personalized learning responses. The project also aligns with the WBS milestones from M1 to M6 and is suitable for final demonstration, documentation, and presentation.